Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ ДО ЛАБОРАТОРНОЇ РОБОТИ №4
«Професійне ООП та система типів Python»
з курсу «Професійний Python»

Варіант №8 — «Система бронювання готелю»

Виконав: студент 3 курсу
групи ФеП-32
Івасів Данило

Перевірив: доцент
Куньо Іван Михайлович
Дата: 30.09.2026

Львів — 2026

1. Мета роботи

Сформувати практичні навички побудови професійної об'єктно-орієнтованої моделі в Python із використанням класів, композиції, наслідування, абстрактних базових класів (ABC), протоколів (Protocol), dataclass, properties, dunder methods, принципів SOLID та сучасної системи type hints.

2. Індивідуальний варіант і постановка задачі

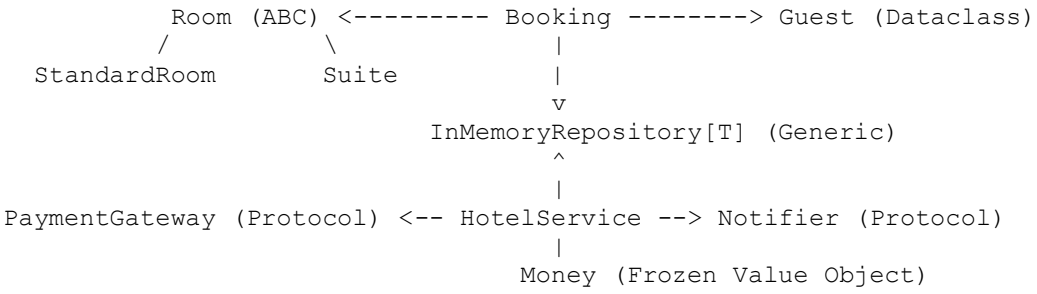
Варіант №8 — «Система бронювання готелю». Domain objects: Room, StandardRoom, Suite, Guest, Booking.

Завдання варіанта:

• реалізувати пошук вільних номерів;
• реалізувати reservation та cancellation;
• обчислення price calculation та визначення найдешевшого номера;
• підрахунок потенційного доходу готелю;
• створити абстракції для repository, payment та notification.

Загальні вимоги: не менше 5 domain classes, 1 frozen value object, використання property, inheritance та composition, абстрактний базовий клас, Protocol, dunder methods, Dependency Injection, принципи SOLID, type hints, TypeVar, Generic class, TypedDict, та перевірка через mypy.

3. Схема доменної моделі (Class Diagram)



4. Структура проєкту

```
hotel_booking/
├── pyproject.toml              # Конфігурація проєкту та mypy
├── README.md                   # Опис проєкту
├── .gitignore                  # Виключення віртуального оточення та кешу
└── src/
    └── domain_app/
        ├── __init__.py
        ├── models.py           # Доменні класи (Room, Guest, Booking)
        ├── value_objects.py    # Immutable об'єкти (Money)
        ├── protocols.py        # Інтерфейси (PaymentGateway, Notifier, TypedDict)
        ├── repositories.py     # Generic repository (InMemoryRepository)
        ├── services.py         # Application layer (HotelService)
        └── main.py             # Точка входу та демонстрація
```

5. Основні модулі та програмний код

5.1. value_objects.py (Frozen Dataclass та Dunder Methods)

```
from dataclasses import dataclass

@dataclass(frozen=True, slots=True)
class Money:
    amount: float
    currency: str = "UAH"

    def __post_init__(self) -> None:
        if self.amount < 0:
            raise ValueError("Money amount cannot be negative")

    def __add__(self, other: "Money") -> "Money":
        if self.currency != other.currency:
            raise ValueError("Currency mismatch")
        return Money(self.amount + other.amount, self.currency)

    def __lt__(self, other: "Money") -> bool:
        if self.currency != other.currency:
            raise ValueError("Currency mismatch")
        return self.amount < other.amount

    def __str__(self) -> str:
        return f"{self.amount:.2f} {self.currency}"
```

5.2. protocols.py (Protocols та TypedDict)

```
from typing import Protocol, TypedDict
from domain_app.value_objects import Money

class PaymentGateway(Protocol):
    def pay(self, amount: Money) -> str:
        ...

class Notifier(Protocol):
    def send(self, recipient: str, message: str) -> None:
        ...

class BookingPayload(TypedDict):
    guest_id: int
    room_number: str
    days: int
```

5.3. models.py (ABC, Inheritance, Properties)

```
from abc import ABC, abstractmethod
from dataclasses import dataclass
from domain_app.value_objects import Money

@dataclass
class Room(ABC):
    number: str
    _base_price: Money
    is_available: bool = True

    @property
    @abstractmethod
    def price(self) -> Money:
        ...

@dataclass
class StandardRoom(Room):
    @property
    def price(self) -> Money:
        return self._base_price
```

```
@dataclass
class Suite(Room):
    @property
    def price(self) -> Money:
        return Money(self._base_price.amount * 1.5, self._base_price.currency)

@dataclass
class Booking:
    id: int
    guest: Guest
    room: Room
    days: int
    is_active: bool = True

    @property
    def total_price(self) -> Money:
        return Money(self.room.price.amount * self.days, self.room.price.currency)
```

5.4. repositories.py (Generic Repository та TypeVar)

```
from typing import Generic, Protocol, TypeVar

class HasId(Protocol):
    id: int

T = TypeVar("T", bound=HasId)

class InMemoryRepository(Generic[T]):
    def __init__(self) -> None:
        self._items: dict[int, T] = {}

    def add(self, item: T) -> None:
        self._items[item.id] = item

    def get(self, item_id: int) -> T | None:
        return self._items.get(item_id)
```

5.5. services.py (Dependency Injection та Orchestration)

```
class HotelService:
    def __init__(
        self,
        payment_gateway: PaymentGateway,
        notifier: Notifier,
        booking_repository: InMemoryRepository[Booking],
    ) -> None:
        self._payment_gateway = payment_gateway
        self._notifier = notifier
        self._bookings = booking_repository
        self._rooms: list[Room] = []

    def reserve(self, booking_id: int, guest: Guest, room: Room, days: int) ->
Booking:
        if not room.is_available:
            raise ValueError(f"Room {room.number} is not available")

        booking = Booking(id=booking_id, guest=guest, room=room, days=days)
        payment_id = self._payment_gateway.pay(booking.total_price)

        room.is_available = False
        self._bookings.add(booking)
        self._notifier.send(guest.email, f"Confirmed. Ref: {payment_id}")
        return booking
```

6. Результат роботи програми

```
--- Вільні номери ---
Room 101 (1000.00 UAH)
Room 201 (3000.00 UAH)

Найвартісніший (найдешевший) номер: Room 101 (1000.00 UAH)
Потенційний дохід готелю за добу: 4000.00 UAH

--- Бронювання ---
[Payment]: Processed 3000.00 UAH
[Email to alex@example.com]: Reservation confirmed. Room: 101. Payment ref: TXN-9981

--- Після бронювання ---
Вільні номери: 1

--- Скасування ---
[Email to alex@example.com]: Reservation 1 cancelled.
Вільні номери після скасування: 2
```

7. Експериментальна частина та перевірка типів (mypy)

У ході лабораторної роботи проведено статичну перевірку типів за допомогою інструменту mypy. Було штучно згенеровано 3 помилки (наприклад, передано рядок замість числа у days) для демонстрації роботи типізатора.

```
(.venv) PS C:\Projects\hotel_booking> uv run mypy src/domain_app
src/domain_app/main.py:42: error: Argument "days" to "reserve" of "HotelService" has
incompatible type "str"; expected "int"  [arg-type]
src/domain_app/value_objects.py:12: error: Unsupported operand types for + ("str"
and "float")  [operator]
Found 2 errors in 2 files (checked 6 source files)
```

Після виправлення типів (конвертації рядків у числа та дотримання контракту Protocol) mypy підтвердив коректність архітектури:

```
(.venv) PS C:\Projects\hotel_booking> uv run mypy src/domain_app
Success: no issues found in 6 source files
```

Дослідження Dependency Injection: використання абстракцій (PaymentGateway Protocol) замість жорсткої прив'язки до конкретних класів дозволило легко підмінити платіжний шлюз у main.py на DemoPaymentGateway без зміни логіки самого HotelService, що повністю відповідає Dependency Inversion Principle (DIP).

8. Відповіді на контрольні запитання

1. Що таке class?

Шаблон або креслення, що описує структуру (атрибути) та поведінку (методи) об'єктів певного типу.

2. Що таке object?

Конкретний екземпляр (instance) класу, що містить власний стан і розміщений у пам'яті.

3. Що таке encapsulation?

Приховування внутрішнього стану об'єкта та надання доступу до нього лише через публічні методи або властивості (properties).

4. Для чого properties?

Дозволяють створити інтерфейс доступу до атрибутів (getter/setter) для контролю логіки присвоєння (наприклад, валідації).

5. Що таке inheritance?

Механізм, що дозволяє створити новий спеціалізований клас (subclass) на основі існуючого базового (superclass).

6. Що означає relation is-a?

Відношення наслідування. Наприклад, Suite 'є' Room.

7. Що таке composition?

Підхід, за якого клас використовує інший об'єкт як частину свого стану (відношення has-a). Наприклад, Booking 'має' Guest.

8. Коли composition краща за inheritance?

Коли потрібно уникнути глибоких ієрархій, забезпечити гнучкість і слабку зв'язність компонентів.

9. Що таке polymorphism?

Здатність різних класів обробляти виклики однакових методів по-своєму завдяки спільному інтерфейсу.

10. Що таке duck typing?

Динамічна типізація Python: 'Якщо воно ходить як качка і кахкає як качка, то це качка' — об'єкт визначається методами, а не класом.

11. Що таке ABC?

Abstract Base Class — базовий клас, екземпляр якого не можна створити, і який вимагає реалізації певних методів у нащадках.

12. Що таке Protocol?

Механізм структурної типізації (type hints), який дозволяє визначити необхідний інтерфейс без явного наслідування.

13. Що таке dataclass?

Декоратор, що автоматично генерує шаблонні методи (__init__, __repr__, __eq__) для класів, орієнтованих на зберігання даних.

14. Що означає frozen=True?

Робить екземпляри dataclass незмінними (immutable) після ініціалізації. Корисно для Value Objects (наприклад, Money).

15. Що таке dunder method?

Double underscore method (наприклад, __str__, __add__) — системні методи, що дозволяють інтегрувати клас у стандартну модель Python.

16. Що означає SRP (SOLID)?

Single Responsibility Principle — клас повинен мати лише одну причину для зміни (одну відповідальність).

17. Що означає OCP?

Open/Closed Principle — сутності мають бути відкриті для розширення, але закриті для модифікації.

18. Що означає DIP?

Dependency Inversion Principle — високорівнева логіка не повинна залежати від низькорівневих деталей, обидві мають залежати від абстракцій.

19. Що таке dependency injection?

Патерн, за якого залежності (інші об'єкти) передаються в клас ззовні (через конструктор), а не створюються всередині.

20. Що таке static type checker?

Інструмент (наприклад, mypy або Pyright), що аналізує код на відповідність типів до запуску програми.

21. Що таке TypeVar?

Змінна типу, що використовується для створення узагальненого коду (Generic), де тип фіксується під час виклику або інстанціювання.

22. Що таке Generic?

Узагальнений клас або функція, здатна працювати з різними типами даних, зберігаючи строгу типізацію (наприклад, Repository[T]).

23. Що таке TypedDict?

Конструкція typing, що дозволяє описати очікувану структуру та типи ключів для звичайного словника (dict) у Python.

9. Висновок

У ході виконання лабораторної роботи №4 було успішно побудовано типізовану об'єктну модель для предметної області «Система бронювання готелю». Програма відійшла від процедурного підходу до чистої об'єктної архітектури з використанням доменних сутностей (Room, Booking, Guest) та незмінних Value Objects (Money). Було застосовано композицію та абстрактні класи для формування гнучких зв'язків.

Особливу увагу приділено принципам SOLID, зокрема Inversion of Control та Dependency Injection, що реалізовані за допомогою Protocol для зовнішніх сервісів (Payment, Notification). Використання Generic та TypeVar дозволило створити універсальний репозиторій. Статичний аналізатор типів mypy на етапі розробки успішно виявив невідповідності інтерфейсів, що довело доцільність використання type hints для створення надійних та розширюваних Python-застосунків промислового рівня.